AIAssistedCoding

Assignment - 02

Name : N.Ananthan

RollNo:2303A54062

Batch: 47_b

Lab2:ExploringAdditionalAICodingToolsbeyondCopilot –Gemini(Colab)and Cursor AI

Task1:StatisticalSummaryforSurveyData

- **Scenario:Youareadataanalystinternworking withsurveyresponsesstoredas numerical lists.**
  - **Promptused:**
    "Write a Python function that takes a list of numbers and returns the mean, minimum, and maximum values."
  - **Generatedcode:**

```
importstatistics
defanalyze_numbers(numbers):
"""
Calculatesthemean,minimum,andmaximumofalistofnumbers. Args:
numbers(list):Alistofnumericalvalues.
Returns:
tuple:Atuplecontaining(mean,minimum,maximum). Returns
(None, None, None) if the list is empty.
"""
ifnot numbers:
return None, None,None
mean_value=statistics.mean(numbers)
min_value = min(numbers)
max_value= max(numbers)
returnmean_value,min_value, max_value
```

Example Usage

```
data=[10,20, 30,40, 50]
mean,minimum,maximum=analyze_numbers(data)
print(f"List: {data}")
print(f"Mean: {mean}")
print(f"Minimum: {minimum}")
print(f"Maximum:{maximum}")
```

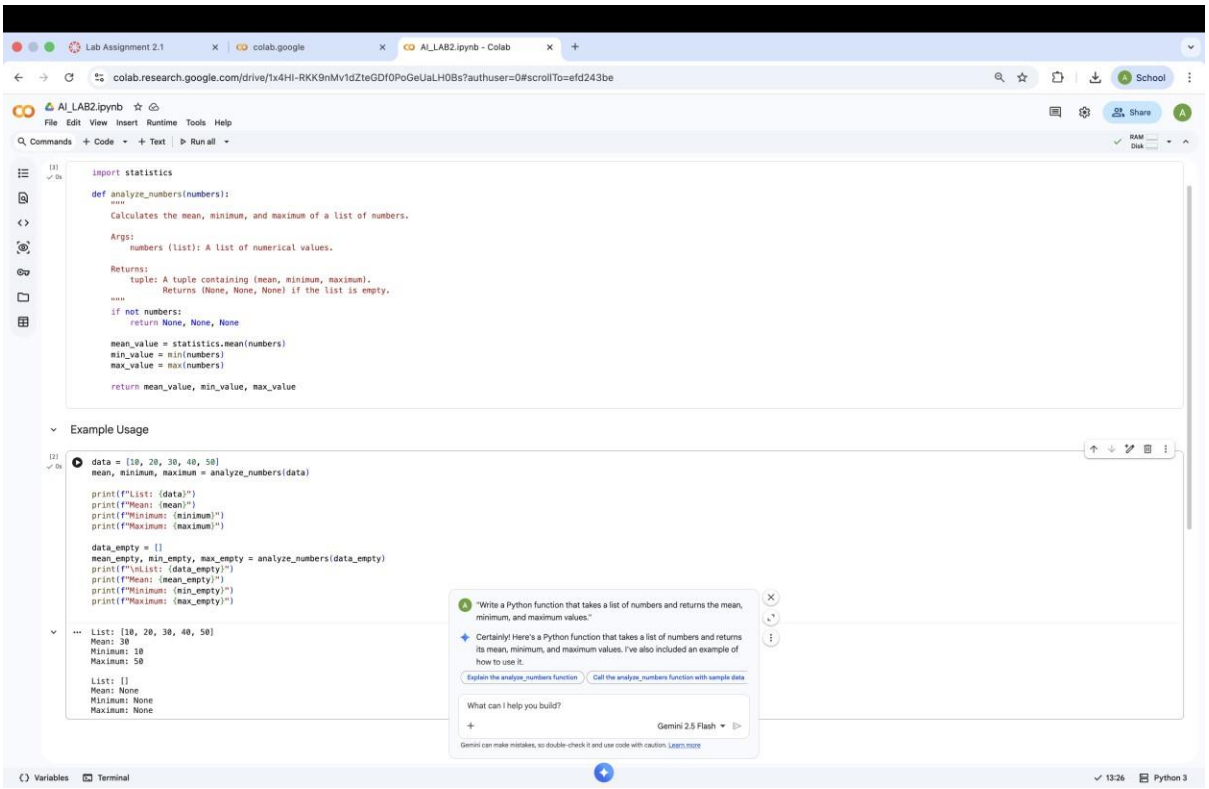
```
data_empty = []
mean_empty,min_empty,max_empty=analyze_numbers(data_empty)
print(f"\nList: {data_empty}")
print(f"Mean: {mean_empty}")
print(f"Minimum: {min_empty}")
print(f"Maximum:{max_empty}")
```

- **SampleOutput:**
  List:[10,20,30,40,50]Mean:30Minimum:10Maximum:50 List: []
  Mean: None Minimum: None Maximum: None
- **ShortExplanationofLogic:**

  The **mean** represents the average of a set of numbers. It is obtained by adding all the values in the list and dividing the sum by the total number of elements.

  The**minimumvalue**isthesmallestnumberpresentinthelist,andthe**maximumvalue** is the largest number present in the list. These values are calculated using Python's built-in min() and max() functions.

Thefunctionacceptsalistofnumericalvaluesasinput,computesthemean,minimum, and maximum values, and displays them as a statistical summary of the given data.



Task2:ArmstrongNumber–AI Comparison

❖ **Scenario:YouareevaluatingAItoolsfornumericvalidation logic.**

- **Promptused(Gemini):**

"WriteaPythonprogramtocheckwhetheragivennumberis anArmstrongnumber."

- **Promptused(Copilot):**
  "WriteaPythonprogramtocheckwhetheragivennumberis anArmstrongnumber."
- **Generatedcode(Gemini):**

```
defis_armstrong_number(number):
"""
Checksifa givennumber isanArmstrongnumber.
AnArmstrongnumber(alsoknownasanarcissisticnumber,
pluperfect digital invariant, or plus perfect number)
isanumberthatisthesumofitsowndigitseachraised to the
power of the number of digits.
rgs:
number(int):Theintegertocheck. Returns:
bool:TrueifthenumberisanArmstrongnumber,Falseotherwise. """
ifnotisinstance(number,int)ornumber<0: return
False
num_str=str(number) n
= len(num_str)
sum_of_powers = 0
for digit_char in num_str:
digit = int(digit_char)
sum_of_powers+=digit**n
returnsum_of_powers== number
```

- **Generatedcode(Copilot):**

```
#!/usr/bin/envpython3"""Armstrong
numberchecker(task2).

Usage:
python3task2.py153
python3task2.py#promptsforinput"""
importargparsei
mportsys



defis_armstrong(n: int)->bool:
if n<0:
returnFalse
digits=[int(d)fordinstr(n)]p =
len(digits)
return sum(d**pfor d indigits) ==n



defmain() ->None:
parser=argparse.ArgumentParser(description='CheckArmstrongnumbers')par
ser.add_argument('number', nargs='?', type=int, help='integer to check')args =
parser.parse_args()
```

```
ifargs.numberisNone:
try:
s=input('Enteranon-negativeinteger:').strip()num =
int(s)
except (EOFError,
ValueError):print('Invalidinput',file=s
ys.stderr)sys.exit(1)
else:
num= args.number

if is_armstrong(num):
print(f"{num}isanArmstrongnumber.")else:
print(f"{num}is notanArmstrongnumber.")


ifname== 'main':main()
```

- **SampleInput(Gemini):**

  #Testwithsomenumbers
  print(f"Is153anArmstrongnumber?{is_armstrong_number(153)}")#Expected:True (1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153)
  print(f"Is 9474 an Armstrong number? {is_armstrong_number(9474)}") # Expected: True (9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474)
  print(f"Is9anArmstrongnumber?{is_armstrong_number(9)}")#Expected:True(9^1 = 9)
  print(f"Is 10 an Armstrong number? {is_armstrong_number(10)}") # Expected: False (1^2 + 0^2 = 1 != 10)
  print(f"Is123anArmstrongnumber?{is_armstrong_number(123)}")#Expected: False

- **SampleInput(Copilot):**

  Enteranon-negativeinteger:343

- **SampleOutput (Gemini):**

  Is 153 an Armstrong number? True
  Is9474anArmstrongnumber?True Is
  9 an Armstrong number? True
  Is 10 an Armstrong number? False
  Is123anArmstrongnumber?False

- **SampleOutput(Copilot):**

  343is notanArmstrongnumber.

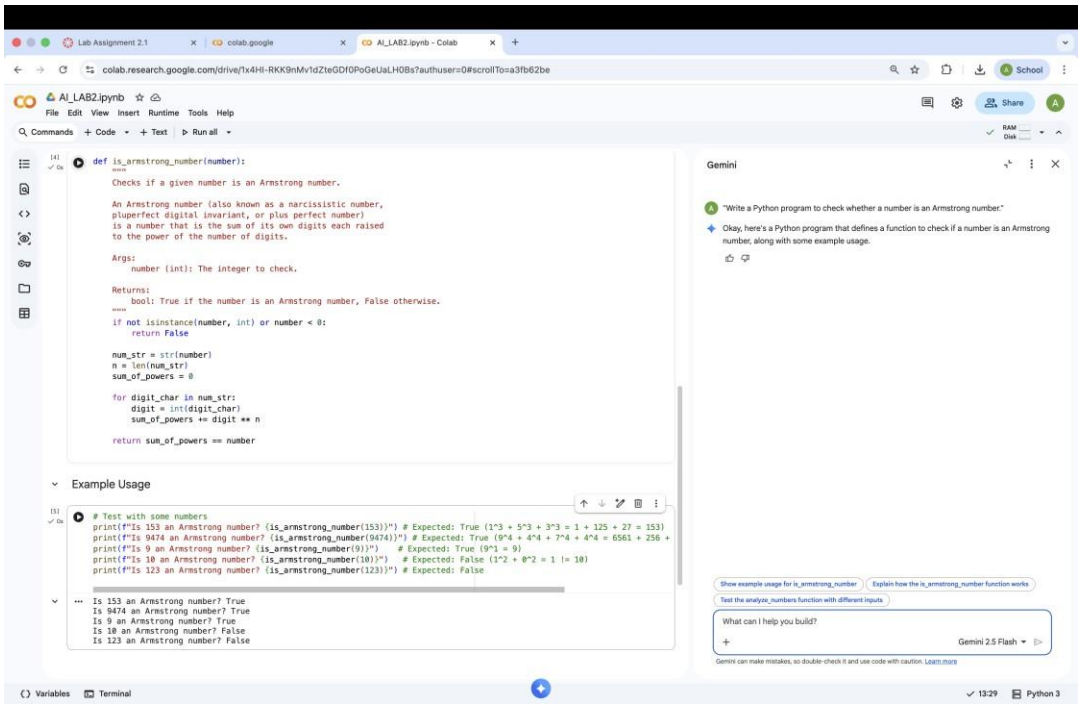
- **Side-by-SideComparisonTable:**

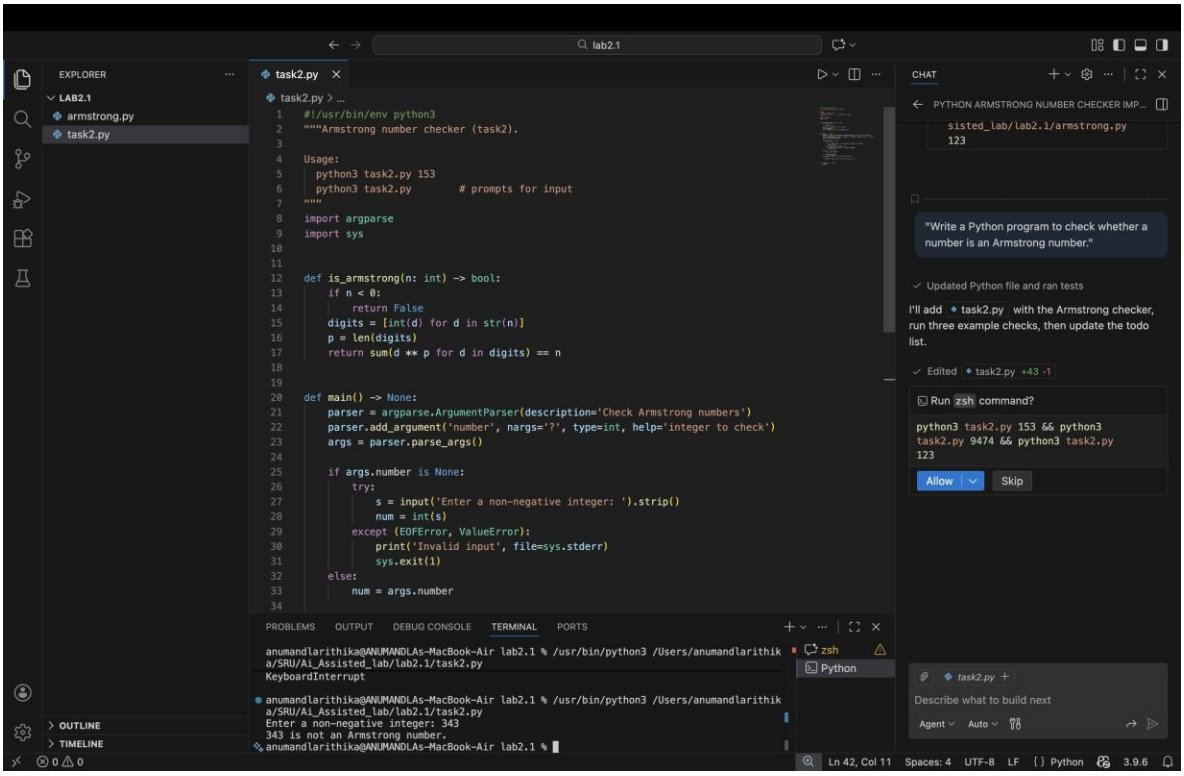
Feature	Google Gemini	GitHub Copilot
Code structure	Uses a separate function	Written directly in main logic
Readability	Very clear and modular	Simple and straightforward
Logic clarity	Easy to understand with function	Easy but less modular
Beginner friendly	High	Medium
Explanation support	Provides explanation along with code	No explanation, only code
Best use case	Learning and documentation	Fast coding inside editor

- **Side-by-SideComparisonTable:**

Analysis : GoogleGeminigenerated awell-structured and modular solution using afunction, making it suitable for learning and reuse. GitHub Copilot produced a concise and direct implementation, which is efficient for quick development within an IDE. Both tools correctly implementedtheArmstrongnumberlogic,butGeminiprovidedbetterclarityandexplanation, while Copilot focused on speed and simplicity.

Conclusion : Both AI tools effectively generated correct Armstrong number checking programs. Google Gemini is more helpful for understanding and learning due to its explanations, while GitHub Copilot is ideal for rapid code generation during development.





Task3:LeapYearValidationUsingCursorAI

❖ **Scenario:You arevalidating acalendarmoduleforabackend system.**

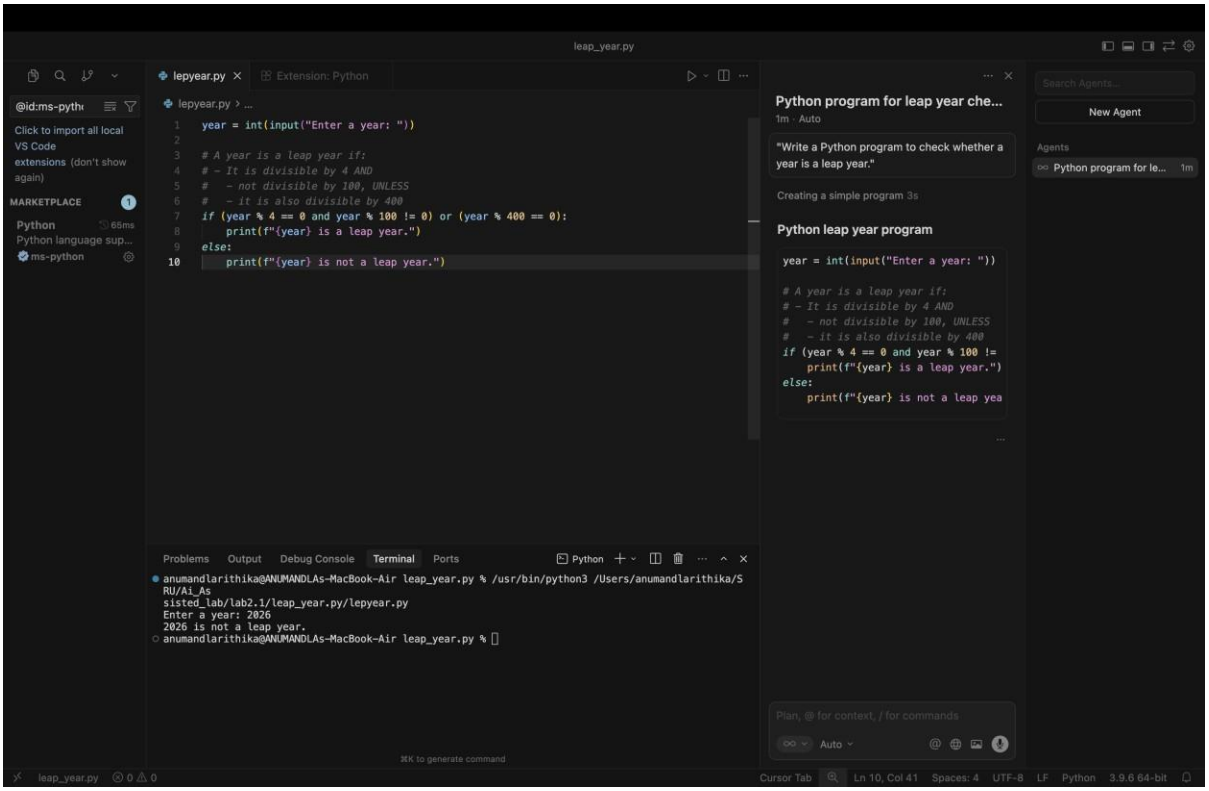
- **Prompt 1:**
  "WriteaPythonprogramtocheckwhether agivenyearis aleap year."
- **Generatedcode:**
  year=int(input("Enterayear:")) #
  A year is a leap year if:
  #-Itis divisibleby4 AND
  #-notdivisibleby100,UNLESS #- it
  is also divisible by 400
  if(year%4==0andyear%100!=0)or(year%400==0): print(f"{year}
  is a leap year.")
  else:
  print(f"{year}isnotaleapyear.")
- **Prompt2:**
  "WriteanoptimizedPythonprogramwithproper conditionsandcommentstocheck whether a year is a leap year."
- **Generatedcode:**

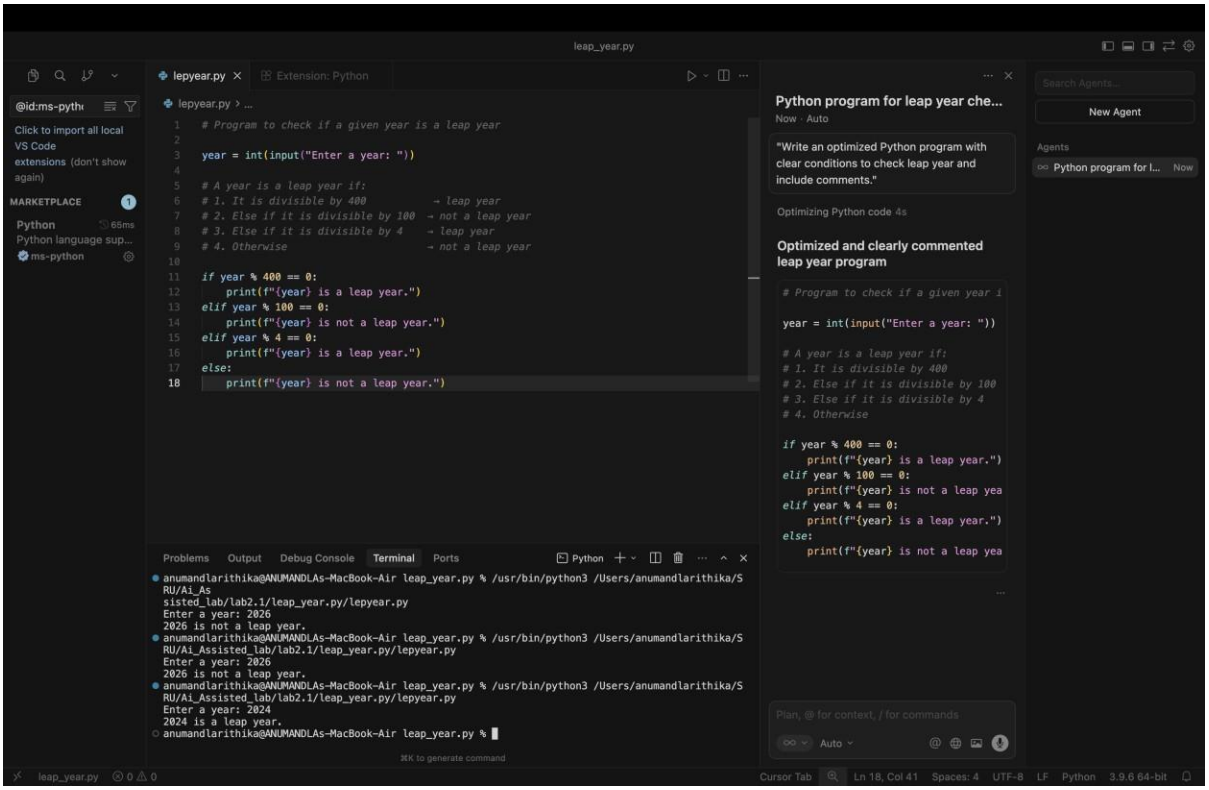
```
year=int(input("Enterayear: "))

#Ayear is aleapyearif:
#1.Itis divisibleby400→leap year
#2.Elseifitisdivisibleby100→notaleapyear# 3.
Else if it is divisible by 4 → leap year
#4.Otherwise→notaleapyear
```

```
if year % 400 ==
0:print(f"{year}isaleapyear.")
elif year % 100 == 0:
print(f"{year}isnotaleap year.")
elif year % 4 ==
0:print(f"{year}isaleapyear.")
else:
print(f"{year}isnotaleap year.")
```

- **SampleInput:**
  Enterayear:2026
- **SampleOutput:**
  2026is notaleap year.
- **ShortExplanationofLogic:**
  ThefirstversionofthecodegeneratedbyCursorAIprovidedbasicleapyearvalidation logic. The second prompt resulted in improved code with better readability and clear commentsexplainingtheconditions.Theoptimizedversioniseasiertounderstand and more suitable for real-world applications.





Task4:StudentLogic+AIRefactoring(Odd/Even Sum)

❖ **Scenario:CompanypolicyrequiresdeveloperstowritelogicbeforeusingAI.**

- **Promptused:**
  "RefactorthisPythoncodetoimprovereadabilityandefficiency."
- **StudentCode:**

  T =(1, 2, 3, 4, 5, 6,7)

  even_sum=0

  odd_sum=0 for

  i in t:

  if i% 2 == 0:

  even_sum=even_sum+i else:

  odd_sum= odd_sum+ i

  print("Sumofevennumbers:",even_sum)

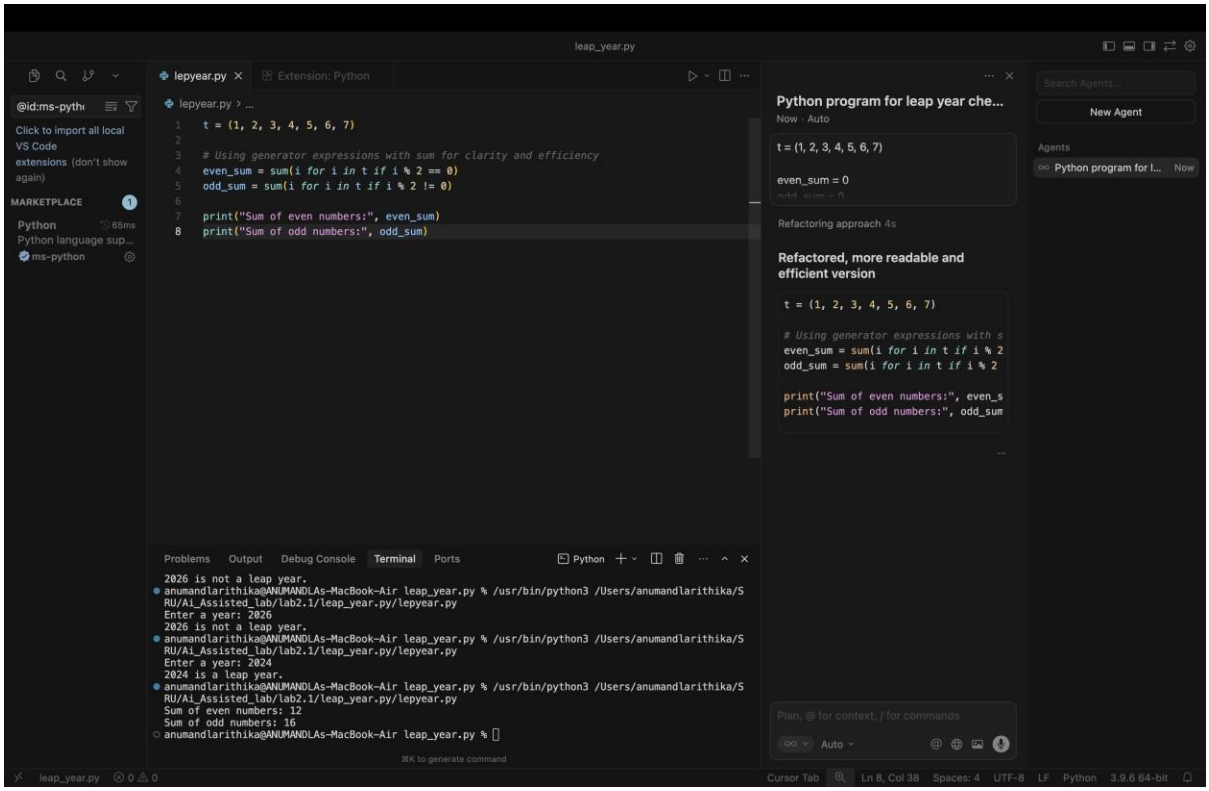
  print("Sum of odd numbers:", odd_sum)

- **AI Code:**

```
t= (1,2,3,4,5,6,7)

#Usinggeneratorexpressionswithsumforclarityand efficiency
even_sum=sum(iforiintifi%2==0)odd_sum =
sum(i for i in t if i % 2 != 0)

print("Sumofevennumbers:",even_sum)print("Sum
of odd numbers:", odd_sum)
```

- **SampleOutput:**

  Sumofevennumbers:12
  Sum of odd numbers: 16